

МИНОБРНАУКИ РОССИИ

Санкт-Петербургский государственный электротехнический
университет «ЛЭТИ» им. В. И. Ульянова (Ленина)

С. А. БЕЛЯЕВ

ИНТЕЛЛЕКТУАЛЬНЫЕ СИСТЕМЫ
Программирование игроков
в виртуальном футболе

Лабораторный практикум



Санкт-Петербург
Издательство СПбГЭТУ «ЛЭТИ»

2020

УДК 004.432

ББК 3 988.02–018я7

Б49

Беляев С. А.

Б49 Интеллектуальные системы. Программирование игроков в виртуальном футболе: лабораторный практикум. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2020. 62 с.

ISBN 978-5-7629-2795-6

Представлены материалы по дисциплине «Интеллектуальные системы». Рассматриваются теоретические вопросы разработки программных агентов и выполняется их практическая отработка в среде виртуального футбола. Приводятся решения по всем этапам от позиционирования игрока на игровом поле и определения координат других объектов до решения тактических и стратегических задач с использованием деревьев решений, временных автоматов и иерархических контроллеров.

Предназначено для студентов направлений «Программная инженерия» и «Прикладная математика и информатика».

УДК 004.432

ББК 3 988.02–018я7

Рецензенты: АО «Научно-инженерный центр СПбЭТУ» (канд. воен. наук, доцент А. И. Вайнтрауб); канд. техн. наук, доцент Е. Н. Шаповалов (АО «НИИ ПС»).

Утверждено

редакционно-издательским советом университета
в качестве учебного пособия

ISBN 978-5-7629-2795-6

© СПбГЭТУ «ЛЭТИ», 2020

ВВЕДЕНИЕ

Лабораторный практикум содержит методические указания к практическим работам по дисциплине «Интеллектуальные системы». Перечень практических работ соответствует рабочей программе дисциплины и включает в себя:

- определение местонахождения игрока на поле;
- движение игрока по маршруту;
- управление несколькими игроками;
- координацию действий игроков;
- решение целевой задачи в условиях противодействия;
- командную игру.

Информационные технологии (операционные системы, программное обеспечение общего и специализированного назначения, информационные справочные системы) и материально-техническая база, используемые при осуществлении образовательного процесса по дисциплине, соответствуют требованиям федерального государственного образовательного стандарта высшего образования. В результате изучения студенты получают необходимые навыки, закрепляют теоретический материал лекций и приобретают требуемые компетенции.

В учебном пособии использована среда для проведения международных соревнований по виртуальному футболу [<https://rcsoccersim.github.io>]. Описанная среда может выступать в качестве основы для разработки интеллектуальных агентов. В учебном пособии приведена последовательность установки и настройки среды, краткое описание стандарта взаимодействия с сервером среды, фрагменты кода для организации взаимодействия между разрабатываемым интеллектуальным агентом и сервером.

Практические работы предназначены для исследования широкого спектра вопросов создания интеллектуальных агентов, начиная с задачи движения, позиционирования, решения простейших задач до элементов принятия решений и функционирования в мультиагентной среде.

Виртуальный футбол предполагает использование сервера, монитора (средство визуализации ситуации на игровом поле) и 22 программ (интеллектуальных агентов), выступающих в роли игроков двух команд, по 11 в каждой. Основная задача сервера – сбор информации о действиях всех игроков и предоставление каждому из них данных о состоянии окружающего мира.

Важно, что сервер стремится предоставить информацию, максимально приближенную к реальному миру, поэтому в нее закладывается погрешность по координатам объектов, есть ограничения видимости (чем дальше находится объект, тем меньше о нем информации), ограничения по передаче аудиоинформации. Монитор обеспечивает визуализацию процесса игры на плоскости и передачу основных команд, например «Kick off» (вбрасывание), «Play on» (начало игры). Интеллектуальные агенты (игроки) подключаются к серверу по протоколу UDP и могут быть написаны на любом языке программирования. В рамках учебного пособия все примеры приведены на языке JavaScript.

Разработчики среды виртуального футбола предоставляют инструмент просмотра журнала игры, но работа с данным журналом выходит за рамки учебного пособия. С подробным описанием взаимодействия с сервером (отправляемые команды и получаемые ответы) можно ознакомиться в Интернете по адресу: <https://github.com/rcsoccersim/manual>.

1. УСТАНОВКА И НАСТРОЙКА СРЕДЫ РАЗРАБОТКИ

В данном разделе приведены установка и настройка виртуального футбола (два альтернативных варианта).

Установка из исходных кодов на примере ОС Ubuntu 18.04 LTS.
Прежде всего следует установить git для получения сервера/и монитора с GitHub и Node.JS для развертывания клиента:

```
sudo apt-get update
sudo apt-get install -y curl git
curl -sL https://deb.nodesource.com/setup_10.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Затем следует выбрать папку для хранения и загрузить rcsoccersim из GitHub:

```
git clone https://github.com/rcsoccersim/rcssserver.git
git clone https://github.com/rcsoccersim/rcssmonitor.git
```

Затем необходимо настроить Ubuntu для компиляции и выполнения сервера и монитора виртуального футбола:

```
sudo apt-get update
sudo apt-get install -y build-essential automake libboost-all-dev flex bison libqt4-dev libaudio-dev libxt-dev libxi-dev libxrender-dev libfreetype6-dev libfontconfig1-dev
```

Далее следует настройка и сборка сервера:

```
cd rcssserver
./bootstrap
```



```
./configure CXXFLAGS='-std=c++14'
sudo make
sudo make install
```

Затем необходимо выполнить конфигурирование:

```
sudo nano /etc/ld.so.conf
```

В открывшемся для редактирования файле добавить в конец следующую строку:

```
/usr/local/share
```

Затем нажать комбинацию клавиш для сохранения (Ctrl + o, <Enter>) и для выхода из редактора (Ctrl + x), после чего выполнить конфигурирование:

```
sudo ldconfig
```

Запуск сервера осуществляется следующей командой:

```
rcssserver
```

Затем следует перейти в папку rcssmonitor и повторить команды настройки:

```
./bootstrap
./configure CXXFLAGS='-std=c++14'
sudo make
sudo make install
```

Запуск монитора выполняется следующей командой:

```
rcssmonitor
```

Установка «сборки» на примере ОС Ubuntu 20.04 LTS. Прежде всего следует установить Node.JS для развертывания клиента:

```
sudo apt-get update
sudo apt-get install -y curl
curl -sL https://deb.nodesource.com/setup_10.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Затем следует установить пакеты для работы сервера и монитора:

```
sudo add-apt-repository ppa:rock-core/qt4
sudo apt-get update
sudo apt-get install -y g++ build-essential libboost-all-dev
qt4-dev-tools qt5-default libfontconfig1-dev libaudio-dev libxt-
dev libgtk-3-dev libglib2.0-dev libxi-dev libxrender-dev libxt-
dev bison flex
```

Необходимо скачать и распаковать «сборки» (release) сервера и монитора.

Адрес для скачивания сервера:

```
https://github.com/rcsoccersim/rcssserver/releases
```

Адрес для скачивания монитора:

```
https://github.com/rcsoccersim/rcssmonitor/releases
```

Команда для распаковки на примере «сборки» 16.0.0:

```
tar -zxpf rcssserver-16.0.0.tar.gz
tar -zxpf rcssmonitor-16.0.0.tar.gz
```

Дальнейшая настройка сервера (в папке `rcssserver`):

```
./configure && make  
sudo make install
```

Затем необходимо выполнить конфигурирование для работы сервера:

```
sudo nano /etc/ld.so.conf
```

В открывшемся для редактирования файле добавить в конец следующую строку:

```
/usr/local/share
```

Затем нажать комбинацию клавиш для сохранения (`Ctrl + o`, `<Enter>`) и для выхода из редактора (`Ctrl + x`), после чего выполнить конфигурирование:

```
sudo ldconfig
```

Запуск сервера осуществляется следующей командой:

```
rcssserver
```

Дальнейшая настройка монитора (в папке `rcssmonitor`):

```
./configure && make  
sudo make install
```

Запуск монитора выполняется следующей командой:

```
rcssmonitor
```

Последовательность действий для проведения эксперимента:

1. Запуск сервера (`rcssserver`).
2. Запуск монитора (`rcssmonitor`).
3. Запуск программы для агентов обеих команд.
4. В меню программы монитора необходимо выбрать пункт «Kick off», на сервер отправляется соответствующая команда, и начинается игра.

Дополнительно для проведения экспериментов можно загрузить готовые команды, участвующие в международных соревнованиях по виртуальному футболу: <http://archive.robocup.info/Soccer/Simulation/2D/binaries/RoboCup/>, в частности за 2019 год в папке `MainRound` можно загрузить и запустить команды `cygus` и `fraunited`, но это выходит за рамки настоящего лабораторного практикума.

2. ИНТЕРФЕЙС ВЗАИМОДЕЙСТВИЯ С СЕРВЕРОМ

2.1. Введение

Взаимодействие с сервером осуществляется по протоколу UDP. Для отправки сообщений используется порт 6000. Предусмотрены команды подключения и отключения от сервера.

После подключения к серверу каждый такт игры (10 раз в секунду) сервер присылает сообщения о состоянии окружающего мира, в том числе ин-

формацию о том, что воспринимает игрок (видит, слышит, чувствует). У игрока есть возможность отправлять сообщения с командами управления. Большинство команд могут отправляться только один раз за такт. Игрок может игнорировать или «пропускать» такты игры, но игра при этом не останавливается, а продолжается в реальном масштабе времени.

Соревнования по виртуальному футболу предполагают, что игроки действуют полностью автономно. Программное взаимодействие между ними запрещено правилами игры. Единственный допустимый вариант – взаимодействие через сервер с использованием аудиосообщений, которые имеют свои ограничения.

2.2. Подключение и отключение от сервера

Подключение игрока к серверу осуществляется командой следующего вида:

```
(init TeamName [(version Version)] [(goalie)])
```

В игре могут принять участие только две команды, поэтому при подключении игроков в качестве TeamName могут использоваться только два имени (названия первой и второй команд). Текст в квадратных скобках – не обязательный, но следует понимать, что при отсутствии указания на версию взаимодействие с сервером будет осуществляться по максимальной доступной версии. В учебном пособии все примеры будут описаны для версии № 7. Здесь goalie означает регистрацию вратаря. Ответ от сервера:

```
(init Side Unum PlayMode)
```

Side принимает значения «l» (команда слева) или «r» (команда справа), Unum – номер игрока в команде, PlayMode – текущий режим игры. Примеры UDP-сообщений запроса к серверу и ответа от него:

```
(init MyTeam (version 7))
```

```
(init 1 2 before_kick_off)
```

Команда отключения от сервера:

```
(bye)
```

2.3. Получение данных от сервера игры

1. Сообщения о том, что ощущает игрок. Ощущения от тела игрока предоставляются в сообщении (sense_body) в следующем формате:

```
(sense body Time  
  (view mode {high | low} {narrow | normal | wide} )  
  (stamina Stamina Effort)  
  (speed AmountOfSpeed DirectionOfSpeed)
```



```

(head_angle HeadAngle)
(kick KickCount)
(dash DashCount)
(turn TurnCount)
(say SayCount)
(turn_neck TurnNeckCount)
(catch CatchCount)
(move MoveCount)
(change_view ChangeViewCount)
)

```

Здесь после открытой скобки пишется название сообщения или свойства, затем – либо числовое значение (Time, Stamina, Effort и т. д.), либо строковое значение (high, low, narrow и т. д.). В фигурных скобках через знак «|» перечисляются альтернативные значения.

Здесь Time – имитируемое время игры (номер такта игры), Stamina – количество энергии у игрока (положительное вещественное число), Effort – усилие (положительное вещественное число), AmountOfSpeed – скорость (положительное вещественное число), DirectionOfSpeed – направление скорости (от -180 до 180°), HeadAngle – поворот головы (от -180 до 180°), все Count – целое число.

Примеры сообщений от сервера:

```

(sense_body 4 (view_mode high normal) (stamina 8000 1) (speed
0.02 -34) (head_angle 0) (kick 0) (dash 1) (turn 3) (say 0)
(turn_neck 0) (catch 0) (move 1) (change_view 0))
(sense_body 5 (view_mode high normal) (stamina 7945 1) (speed
0.24 -4) (head_angle 0) (kick 0) (dash 2) (turn 3) (say 0)
(turn_neck 0) (catch 0) (move 1) (change_view 0))
(sense_body 6 (view_mode high normal) (stamina 7890 1) (speed
0.35 0) (head_angle 0) (kick 0) (dash 3) (turn 3) (say 0)
(turn_neck 0) (catch 0) (move 1) (change_view 0))
(sense_body 7 (view_mode high normal) (stamina 7935 1) (speed
0.15 -5) (head_angle 0) (kick 0) (dash 3) (turn 4) (say 0)
(turn_neck 0) (catch 0) (move 1) (change_view 0))

```

В сообщениях видно изменение времени с 4 до 7, изменение энергии игрока stamina, изменение скорости speed, команды на ускорение dash, угла поворота turn.

2. Сообщения о том, что видит игрок. Сообщения see о том, что видит игрок, предоставляются в следующем формате:

```
(see Time ObjInfo+)
```

Здесь Time – счетчик времени, ObjInfo – информация о видимом объекте (таких сообщений может быть множество). Формат ObjInfo:

```

ObjInfo ::= (ObjName Distance Direction DistChange DirChange
BodyFacingDir HeadFacingDir)

```

```

| (ObjName Distance Direction DistChange DirChange)
| (ObjName Distance Direction)
| (ObjName Direction)

```

Здесь ObjName – имя объекта, Distance – расстояние до объекта (положительное вещественное число), Direction – направление (от -180 до 180°), DistChange и DirChange – изменение дистанции и направления (вещественное число), HeadFaceDir и BodyFaceDir – направление головы и лица (от -180 до 180°). При этом отрицательный угол означает, что объект находится слева, положительный – справа. Количество предоставляемой информации зависит от расстояния до объекта (чем больше расстояние, тем меньше информации доступно). ObjName имеет следующий формат:

```

ObjName ::= (p ["Teamname" [UniformNumber [goalie]]])
| (b) | (g [l|r]) | (f c) | (f [l|c|r] [t|b])
| (f p [l|r] [t|c|b]) | (f g [l|r] [t|b])
| (f [l|r|t|b] 0) | (f [t|b] [l|r] [10|20|30|40|50]) |
| (f [l|r] [t|b] [10|20|30])
| (l [l|r|t|b]) | (B) | (F) | (G) | (P)

```

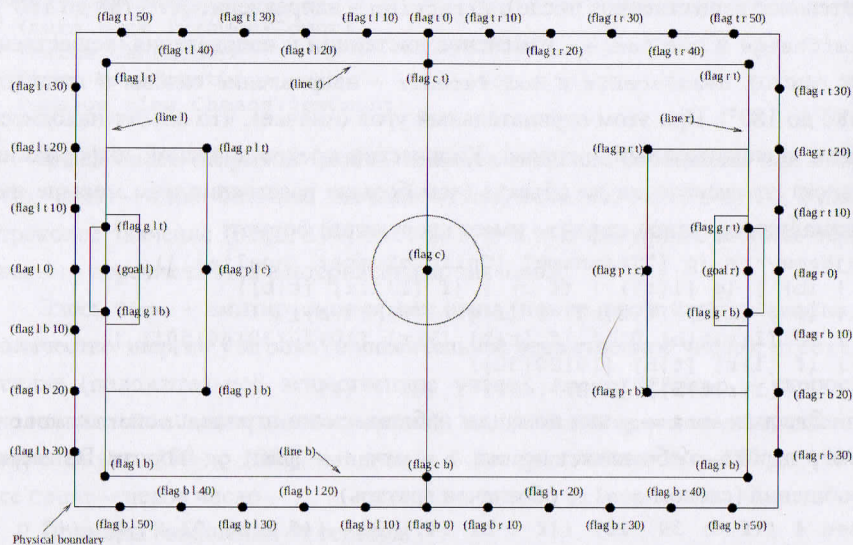
Здесь Teamname – имя команды наблюдаемого игрока, UniformNumber – номер игрока, p обозначает игрока, b – мяч, f – флаг, g – ворота. Примеры сообщений (словом goalie обозначен вратарь):

```

(see 4 ((f c) 30 -13) ((f c b) 54.1 22) ((f r b) 92.8 -8) ((f g
r b) 82.3 -24) ((g r) 80.6 -29) ((f g r t) 79.8 -34) ((f p r b)
71.5 -11) ((f p r c) 64.1 -27) ((f p r t) 63.4 -45) ((f b 0)
58.6 24) ((f b r 10) 64.1 16) ((f b r 20) 70.1 10) ((f b r 30)
77.5 4) ((f b r 40) 84.8 0) ((f b r 50) 92.8 -4) ((f b l 10)
54.6 34) ((f b l 20) 52.5 44) ((f r 0) 85.6 -30) ((f r t 10)
84.8 -36) ((f r t 20) 84.8 -43) ((f r b 10) 87.4 -23) ((f r b
20) 90.9 -17) ((f r b 30) 94.6 -11) ((b) 30 -13) ((l b) 75.9 -
38))
(see 6 ((f c) 28.5 -13 -0.57 -0.2) ((f c b) 53 22) ((f r b) 90.9
-8) ((f g r b) 80.6 -25) ((g r) 79 -29) ((f g r t) 78.3 -35) ((f
p r b) 70.1 -11) ((f p r c) 62.8 -27) ((f b 0) 57.4 25) ((f b r
10) 62.2 17) ((f b r 20) 68.7 10) ((f b r 30) 75.9 4) ((f b r
40) 83.1 0) ((f b r 50) 91.8 -4) ((f b l 10) 53.5 34) ((f r 0)
83.9 -30) ((f r t 10) 83.1 -37) ((f r t 20) 83.9 -44) ((f r b
10) 86.5 -23) ((f r b 20) 89.1 -17) ((f r b 30) 93.7 -11) ((b)
30 -13 -0.6 -0.2) ((l b) 74.4 -38))
(see 7 ((f c) 28.2 -20 -0 -0.1) ((f c b) 52.5 16) ((f r b) 90.9
-15) ((f g r b) 80.6 -31) ((g r) 79 -36) ((f g r t) 78.3 -42)
((f p r b) 69.4 -18) ((f p r c) 62.8 -34) ((f b 0) 56.8 18) ((f
b r 10) 62.2 10) ((f b r 20) 68 3) ((f b r 30) 75.2 -3) ((f b r
40) 83.1 -7) ((f b r 50) 90.9 -11) ((f b l 10) 53 28) ((f b l
20) 50.9 39) ((f r 0) 83.9 -37) ((f r t 10) 83.1 -44) ((f r b
10) 85.6 -30) ((f r b 20) 89.1 -24) ((f r b 30) 92.8 -18) ((b)
27.1 -20) ((l b) 64.7 -45))

```


Расположение флагов на поле представлено на рисунке. Далее приведен фрагмент кода на JavaScript константы с координатами флагов. В качестве имен свойств использованы имена флагов (в сообщении see "(f c)",



Игровое поле

флаг fc: {x: 0, y: 0} в центре поля с координатами (0,0)). Для удобства отладки рекомендуется в описание каждого объекта помещать название флага:

```
const Flags = {
  ftl50: {x: -50, y: 39}, ftl40: {x: -40, y: 39},
  ftl30: {x: -30, y: 39}, ftl20: {x: -20, y: 39},
  ftl10: {x: -10, y: 39}, ft0: {x: 0, y: 39},
  ftr10: {x: 10, y: 39}, ftr20: {x: 20, y: 39},
  ftr30: {x: 30, y: 39}, ftr40: {x: 40, y: 39},
  ftr50: {x: 50, y: 39}, fbl50: {x: -50, y: -39},
  fbl40: {x: -40, y: -39}, fbl30: {x: -30, y: -39},
  fbl20: {x: -20, y: -39}, fbl10: {x: -10, y: -39},
  fb0: {x: 0, y: -39}, fbr10: {x: 10, y: -39},
  fbr20: {x: 20, y: -39}, fbr30: {x: 30, y: -39},
  fbr40: {x: 40, y: -39}, fbr50: {x: 50, y: -39},
  flt30: {x: -57.5, y: 30}, flt20: {x: -57.5, y: 20},
  flt10: {x: -57.5, y: 10}, fl0: {x: -57.5, y: 0},
  flb10: {x: -57.5, y: -10}, flb20: {x: -57.5, y: -20},
  flb30: {x: -57.5, y: -30}, frt30: {x: 57.5, y: 30},
  frt20: {x: 57.5, y: 20}, frt10: {x: 57.5, y: 10},
  fr0: {x: 57.5, y: 0}, frb10: {x: 57.5, y: -10},
```

```

frb20: {x: 57.5, y: -20}, frb30: {x: 57.5, y: -30},
fglt: {x:-52.5, y: 7.01}, fgltb: {x:-52.5, y:-7.01},
gl: {x:-52.5, y: 0}, gr: {x: 52.5, y: 0}, fc: {x: 0, y: 0},
fplt: {x: -36, y: 20.15}, fplc: {x: -36, y: 0},
fplb: {x: -36, y:-20.15}, fgtr: {x: 52.5, y: 7.01},
fgrb: {x: 52.5, y:-7.01}, fprr: {x: 36, y: 20.15},
fprc: {x: 36, y: 0}, fprb: {x: 36, y:-20.15},
flt: {x:-52.5, y: 34}, fct: {x: 0, y: 34},
ftr: {x: 52.5, y: 34}, flb: {x:-52.5, y: -34},
fcb: {x: 0, y: -34}, frb: {x: 52.5, y: -34},
distance(p1, p2) {
    return Math.sqrt((p1.x-p2.x)**2+(p1.y-p2.y)**2)
},
}

```

Вычисление параметров ObjName осуществляется по следующим формулам:

$$\begin{aligned}
 p_{rx} &= r_{xt} - p_{x0}; \\
 p_{ry} &= r_{yt} - p_{y0}; \\
 v_{rx} &= v_{xt} - v_{x0}; \\
 v_{ry} &= v_{yt} - v_{y0}; \\
 \text{Distance} &= \sqrt{p_{rx}^2 + p_{ry}^2}; \\
 \text{Direction} &= \arctan\left(\frac{p_{ry}}{p_{rx}}\right) - a_0; \\
 e_{rx} &= p_{rx} / \text{Distance}; \\
 e_{ry} &= p_{ry} / \text{Distance}; \\
 \text{DistChng} &= (v_{rx} * e_{rx}) + (v_{ry} * e_{ry}); \\
 \text{DirChng} &= \frac{-(v_{ry} * e_{ry}) + (v_{rx} * e_{rx})}{\text{Distance}} * \frac{180}{\pi}; \\
 \text{BodyDir} &= \text{PlayerBodyDir} - \text{AgentBodyDir} - \text{AgentHeadDir}; \\
 \text{HeadDir} &= \text{PlayerHeadDir} - \text{AgentBodyDir} - \text{AgentHeadDir}.
 \end{aligned}$$

Здесь p_{xt}, p_{yt} – абсолютное положение целевого объекта; p_{x0}, p_{y0} – абсолютное положение игрока; v_{xt}, v_{yt} – абсолютная скорость объекта; v_{x0}, v_{y0} – абсолютная скорость игрока; a_0 – абсолютное направление, куда смотрит игрок, вычисляется как сумма BodyDir и HeadDir игрока. Дополнительно p_{rx}, p_{ry} и v_{rx}, v_{ry} – относительные позиция и скорость цели; (e_{rx}, e_{ry}) – единичный вектор, параллельный вектору относительного положения. BodyDir и

HeadDir предоставляются только для игроков. Таким образом, игрок смотрит прямо, если BodyDir=0. То же относится к HeadDir.

Пример сообщений от сервера до удара по мячу и после удара по мячу (сила удара равна 100):

```
(see 1399 ((f p r t) 1.4 4 -0 -0) ((f t r 10) 33.1 -40 -0 -0)
((f t r 20) 25.8 -26) ((f t r 30) 21.1 -5 -0 -0) ((f t r 40)
20.5 23 -0 0) ((b) 1.3 4 -0 -0) ((l t) 15.6 -78))
(see 1402 ((f p r t) 0.5 12 -0.09 2.1) ((f t r 10) 32.5 -41 -0 -
0.1) ((f t r 20) 25 -27 -0 -0.1) ((f t r 30) 20.3 -5 -0 -0) ((f
t r 40) 19.9 24 -0 0.1) ((b) 0.5 12 -0.09 2.1) ((l t) 14.7 -78))
(see 1403 ((f p r t) 0.5 15 -0.04 1.5) ((f t r 10) 32.5 -41 -0 -
0) ((f t r 20) 25 -27 -0 -0) ((f t r 30) 20.3 -5 -0 -0) ((f t r
40) 19.7 24 -0 0.1) ((b) 0.4 15 -0.032 1.5) ((l t) 14.6 -78))
(see 1405 ((f p r t) 0.4 17 -0.008 0.3) ((f t r 10) 32.5 -41 -0
-0) ((f t r 20) 25 -27) ((f t r 30) 20.1 -5 -0 -0) ((f t r 40)
19.7 24 -0 0) ((b) 0.4 17 -0.008 0.3) ((l t) 14.6 -78))
```

В приведенном примере увеличивается расстояние до мяча (b) и наблюдается погрешность в определении направлений и углов.

Следует обратить внимание, что сервер вычисляет все значения точно, но перед передачей информации пользователю добавляет «шум», соответственно, значения, полученные в see, отличаются от точных значений.

3. Сообщения о том, что слышит игрок. Аудиосообщения для игрока предоставляются в сообщении hear в следующем формате:

```
(hear Time Sender "Message")
```

Здесь Time – время сообщения, Sender – отправитель: online_coach_left | online_coach_right | coach | referee | self | Direction. Direction – направление источника звука (от -180 до 180°).

Пример сообщения:

```
(hear 0 referee kick_off_1)
(hear 15 self "kick")
(hear 1345 referee drop_ball)
(hear 1345 referee play_on)
```

2.4. Команды управления

1. Команды до начала игры. До начала игры или после гола игроки могут быть расставлены на поле с использованием команды move:

```
(move x y)
```

Координаты (x, y) могут принимать значения из следующих интервалов: x – от минус 54 до 54, y – от минус 32 до 32. Знак по x зависит от стороны – левая или правая. Пример для команды слева – игрок находится прямо напротив мяча (в центре поля):

(move -15 0)

2. Команды в процессе игры. Команда turn – повернуться:

(turn Moment)

Здесь Moment значение от -180 до 180° . При этом будет выполнен разворот в заданном направлении относительно текущего положения тела игрока.

Команда dash – бегать:

(dash Power)

Команда дает ускорение игроку в направлении тела (а не текущего направления вектора скорости). Здесь Power – сила от minpower (-100) до maxpower (100).

Команда kick – пнуть мяч:

(kick Power Direction)

Выполняет удар по мячу с силой Power в направлении Direction. Направление Direction вычисляется относительно текущего направления тела игрока, Power – прилагаемая сила от minpower до maxpower.

Команда catch – поймать мяч:

(catch Direction)

Выполняется попытка поймать мяч с направления Direction (от -180 до 180°). Эту команду может выполнить только вратарь goalie, расстояние до мяча должно быть меньше 2.

Важно, что в один цикл игры может быть выполнена только одна из перечисленных пяти команд (move, turn, dash, kick, catch). В случае прихода нескольких команд на сервер он будет выполнять одну, выбранную случайным образом.

3. Дополнительные команды. Поворот головы:

(turn_neck Angle)

Здесь Angle – угол поворота (от -90 до 90°).

Передача аудиосообщения (ограничение радиуса – до 50):

(say Message)

Передаваемое сообщение может содержать символы латинского алфавита и/или символы «() . + * / ? < >», при получении текст сообщения будет в кавычках.

2.5. Вопросы для самопроверки

1. На каком языке программирования можно написать программу управления игроком в виртуальном футболе?

2. Сколько времени есть у программы для формирования команды управления игроком?
3. Какой командой осуществляется подключение и отключение от сервера? Есть ли отличия у вратаря при подключении?
4. Какие сообщения приходят от сервера? Опишите состав получаемой информации.
5. Какие команды программа может отправлять на сервер? Есть ли ограничения по количеству отправляемых команд?
6. Как игрок может выяснить, где он находится на игровом поле?

3. ВЗАИМОДЕЙСТВИЕ С СЕРВЕРОМ НА JavaScript

В данном разделе описана реализация клиентского приложения, позволяющего игроку перемещаться по игровому полю и бить по мячу. Реализация выполнена в виде четырех файлов:

- 1) app.js – основное приложение;
- 2) agent.js – игрок;
- 3) socket.js – сокет для взаимодействия с сервером;
- 4) msg.js – разбор сообщений.

3.1. Основное приложение

В основном приложении создается экземпляр агента, настраивается сокет, выполняется размещение игрока на поле до начала игры:

```
const Agent = require('./agent') // Импорт агента
const VERSION = 7 // Версия сервера
let teamName = "teamA" // Имя команды
let agent = new Agent(); // Создание экземпляра агента
require('./socket')(agent, teamName, VERSION) //Настройка сокета
agent.socketSend("move", `-15 0`) // Размещение игрока на поле
```

3.2. Игрок

Игрок поддерживает размещение слева и справа, умеет по команде с консоли выполнять повороты влево, вправо, двигаться вперед и выполнять удар по мячу:

```
const Msg = require('./msg')
// Подключение модуля разбора сообщений от сервера
const readline = require('readline')
// Подключение модуля ввода из командной строки
class Agent {
  constructor() {
```

```

this.position = "l" // По умолчанию - левая половина поля
this.run = false // Игра начата
this.act = null // Действия
this.rl = readline.createInterface({ // Чтение консоли
    input: process.stdin,
    output: process.stdout
})
this.rl.on('line', (input) => { // Обработка строки из кон-
соли
    if(this.run) { // Если игра начата
        // Движения вперед, вправо, влево, удар по мячу
        if("w" == input) this.act = {n: "dash", v: 100}
        if("d" == input) this.act = {n: "turn", v: 20}
        if("a" == input) this.act = {n: "turn", v: -20}
        if("s" == input) this.act = {n: "kick", v: 100}
    }
})
}
msgGot(msg) { // Получение сообщения
    let data = msg.toString('utf8') // Приведение к строке
    this.processMsg(data) // Разбор сообщения
    this.sendCmd() // Отправка команды
}
setSocket(socket) { // Настройка сокета
    this.socket = socket
}
socketSend(cmd, value) { // Отправка команды
    this.socket.sendMsg`(${cmd} ${value})`
}
processMsg(msg) { // Обработка сообщения
    let data = Msg.parseMsg(msg) // Разбор сообщения
    if(!data) throw new Error("Parse error\n" + msg)
    // Первое (hear) - начало игры
    if(data.cmd == "hear") this.run = true
    if(data.cmd == "init") this.initAgent(data.p) // Инициализация
    this.analyzeEnv(data.msg, data.cmd, data.p) // Обработка
}
initAgent(p) {
    if(p[0] == "r") this.position = "r" // Правая половина поля
    if(p[1]) this.id = p[1] // id игрока
}
analyzeEnv(msg, cmd, p) { // Анализ сообщения
}
sendCmd() {
    if(this.run) { // Игра начата
        if(this.act) { // Есть команда от игрока
            if(this.act.n == "kick") // Пнуть мяч
                this.socketSend(this.act.n, this.act.v + " 0")
            else // Движение и поворот
                this.socketSend(this.act.n, this.act.v)
        }
    }
}

```



```

    this.act = null // Сброс команды
  }
}
module.exports = Agent // Экспорт игрока

```

Следует отметить, что для управления игроком из консоли используется модуль `readline`, который читает ввод из консоли по строкам, поэтому при управлении таким игроком необходимо после каждой команды (`w`, `d`, `a`, `s`) нажимать клавишу «Перевод каретки» (`<Enter>`). С точки зрения разрабатываемой игры это, конечно, неприемлемо, но с точки зрения изучения среды для последующего автоматического исполнения вполне допустимо.

Игрок при создании должен быть зарегистрирован на сервере, в методе `initAgent` обрабатывается ответ сервера. Свойство `run` обеспечивает запуск обработки основного потока сообщений от сервера. В данном случае использован упрощенный метод перехода, когда услышана какая-то команда (рефери объявляет начало игры). Функция `analyzeEnv` предназначена для обработки сообщений от сервера. Функция `sendCmd` отправляет следующую команду на сервер, в предложенном примере – движение вперед, поворот и удар по мячу.

Следует обратить внимание, что свойство `act` в приведенном примере – объект, который многократно пересоздается, что недопустимо в условиях экономии памяти для продолжительной игры.

3.3. Сокет для взаимодействия с сервером

Взаимодействие с сервером осуществляется с использованием UDP на порту 6000 с использованием модуля `dgram`, при этом считается, что сервер запущен на том же компьютере, что и агент:

```

const dgram = require('dgram') // Модуль для работы с UDP
module.exports = function(agent, teamName, version) {
  // Создание сокета
  const socket = dgram.createSocket({ type: 'udp4', reuseAddr:
true })
  agent.setSocket(socket) // Задание сокета для агента
  socket.on('message', (msg, info) => {
    agent.msgGot(msg) // Обработка полученного сообщения
  })
  socket.sendMsg = function(msg) { // Отправка сообщения серверу
    socket.send(Buffer.from(msg), 6000, 'localhost', (err,
bytes) => {
      if(err) throw err
    })
  })
}

```

```

}
// Инициализация игрока на сервере (без параметра goalie)
socket.sendMsg(`(init ${teamName} (version ${version}))`)
}

```

3.4. Разбор сообщений

Разбор сообщений может быть реализован различными способами. В данном примере для разбора сообщений использовано регулярное выражение `(\\(|[-\\d\\.]+|\\\\\\"\\w|\\|\\))`, которое позволяет использовать в качестве разделителей пробелы и скобки. Обработка построена на рекурсивном разборе полученного сообщения:

```

module.exports = {
  parseMsg(msg) { // Разбор сообщения
    if(msg.endsWith("\\u0000")) // Удаление символа в конце
      msg = msg.substring(0, msg.length - "\\u0000".length)
    // Разбор сообщения
    let array = msg.match(/\\(|[-\\d\\.]+|\\\\\\"\\w|\\|\\)/g)
    let res = { msg, p: [] } // Результирующее сообщение
    // Анализировать с индекса 0, результат в res
    this.parse(array, { idx: 0 }, res)
    this.makeCmd(res) // Выделить команду
    return res
  },
  parse(array, index, res) { // Разбор сообщения в скобках
    // Всегда с открывающей скобки
    if(array[index.idx] !== "(")
      return
    index.idx++
    // Разбор внутри скобок
    this.parseInner(array, index, res)
  },
  parseInner(array, index, res) {
    // Пока не встретится закрывающая скобка
    while (array[index.idx] !== ")") {
      // Если внутри еще одна скобка
      if(array[index.idx] === "(") {
        let r = { p: [] }
        // Рекурсивный вызов с index
        this.parse(array, index, r)
        res.p.push(r)
      } else {
        // Одиночный параметр
        let num = parseFloat(array[index.idx])
        res.p.push(isNaN(num) ? array[index.idx] : num)
        index.idx++
      }
    }
  },
  index.idx++
}

```



```

    },
    makeCmd(res) { // Выделение команды
      if(res.p && res.p.length > 0) {
        // Первый параметр - команда
        res.cmd = res.p.shift()
        // Выделить команды у параметров
        for(let value of res.p)
          this.makeCmd(value)
      }
    }
  }
}

```

Использование данной функции позволяет получить следующие результаты:

```

msg=(see 1399 ((f p r t) 1.4 4 -0 -0) ((f t r 10) 33.1 -40 -0 -
0) ((f t r 20) 25.8 -26) ((f t r 30) 21.1 -5 -0 -0) ((f t r 40)
20.5 23 -0 0) ((b) 1.3 4 -0 -0) ((l t) 15.6 -78))
res.p=[1399,{"p":[1.4,4,0,0],"cmd":{"p":["f","p","r","t"]}},{"p":
:[33.1,-40,0,0],"cmd":{"p":["f","t","r",10]}}, {"p":[25.8,-
26],"cmd":{"p":["f","t","r",20]}}, {"p":[21.1,-
5,0,0],"cmd":{"p":["f","t","r",30]}}, {"p":[20.5,23,0,0],"cmd":{"
p":["f","t","r",40]}}, {"p":[1.3,4,0,0],"cmd":{"p":["b"]}}, {"p":[
15.6,-78],"cmd":{"p":["l","t"]}}]

```

В данном примере msg – исходное сообщение, res – результат его разбора в функции parseMsg, res.p – массив.

3.5. Вопросы для самопроверки

1. Как осуществляется разбор сообщения, полученного от сервера игры?
2. По какому протоколу осуществляется взаимодействие с сервером?
3. Какие обязательные параметры передаются в сообщении при регистрации игрока?
4. Приведите пример команд move, dash, turn.

4. КРАТКИЕ ОСНОВЫ МУЛЬТИАГЕНТНЫХ СИСТЕМ

4.1. Введение

Агент – это сущность, действующая в некоторой окружающей среде (мире). Интеллектуальный агент действует в соответствии с ситуацией для решения своих задач, изменяет окружающую среду, имеет свои цели, может учиться (не все агенты учатся), делает выбор с учетом ограничений на восприятие и вычислительные ресурсы.

Агент в соответствии с функцией полезности (utility) выбирает, какое действие ему выполнить. Эта функция проецирует состояния среды на веще-

ственные числа, большее значение означает более выгодное действие. Рациональным агентом называют эгоистичного агента, который стремится во всех состояниях окружающей среды максимизировать значение функции полезности. Следует отметить, что при формировании функции полезности обычно используют вознаграждение. Чем более отдаленным является вознаграждение (например, гол в футболе), тем меньшую ценность имеют действия – эффект дисконтирования ценности. Переход среды из одного состояния в другое определяется функцией перехода (transition function). Важно, что в большинстве практических задач функция перехода не является детерминированной, она возвращает вероятность перехода среды в интересующее состояние при выполнении заданного действия.

При проектировании агента учитывают следующие аспекты:

- возможность декомпозиции агента на отдельные взаимодействующие модули, которые могут анализироваться независимо друг от друга;
- представление окружающей среды с использованием состояний или свойств, принимающих целочисленные или вещественные значения;
- наличие и особенности горизонта планирования (не планирующий агент, ограниченный горизонт, неопределенный и бесконечный горизонт);
- наличие неопределенности в выборе действия, в результате действия;
- полностью или частично наблюдаемая окружающая среда (неопределенность восприятия);
- наличие одного или нескольких агентов;
- знания предопределены либо получаются из опыта.

Среда виртуального футбола обеспечивает неопределенность восприятия информации и неопределенность в результатах действий, тем самым имитируя реальный мир. В ней, как и во многих других средах, будущее состояние среды определяется текущим состоянием и действием каждого из агентов. Таким образом, управление в виртуальном футболе может строиться на основании марковского процесса принятия решений (MDP – Markov decision process).

Рациональный агент анализирует окружающую среду и выбирает последовательность действий для решения своих задач по изменению состояния среды. Целенаправленный перевод среды из одного состояния в другое осуществляется путем решения задачи планирования. В большинстве случаев задача планирования может быть сведена к поиску пути в графе.

В ситуации частично наблюдаемой окружающей среды, когда агент не воспринимает мир в целом (как, например, в виртуальном футболе), задача планирования усложняется. В этом случае знание агента об окружающей среде представляет собой убеждение о состоянии мира (belief state). При этом вектор belief state указывает распределение вероятности того, что мир и агент в нем находятся в том или ином состоянии.

4.2. Нормальная, расширенная и характеристическая формы игры

В теории игр в качестве функции полезности часто используют матрицу вознаграждения (нормальная форма игры), где строчки обозначают действия одного игрока, столбцы – действия другого, а в каждой ячейке указываются вознаграждения (или штрафы) за соответствующие действия. При представлении игр в нормальной (стандартной) форме обычно осуществляется анализ ситуации на одно действие вперед. С использованием игр в нормальной форме исследуются стратегии поведения в классических задачах, таких как дилемма заключенного, игра полов, игра в цыпленка, в них агенты должны принять решение, не договариваясь.

В расширенной форме игры представляются в виде деревьев, в которых действия обозначаются взвешенными дугами с указанием вознаграждений (или штрафов) соответствующих игроков. При этом игрок выбирает последовательность действий как движение по дереву.

Характеристическая форма игры предполагает возможность создания коалиций, когда агенты могут договариваться друг с другом для суммарного увеличения значений функций полезности. При этом возникает множество других задач, касающихся вклада каждого из агентов в полученный результат, вариантов деления результата между агентами и т. п.

Все эти формы представления относятся к классической теории игр, предполагают самостоятельное принятие решений каждым из агентов, в случае характеристической формы предусматриваются возможности договариваться и участвовать в игре в составе команд.

4.3. Обучение в мультиагентных системах

Задача машинного обучения заключается в нахождении такой функции, которая будет корректно сопоставлять входным параметрам (восприятие окружающей среды) выходные параметры (действия), в том числе для при-

меров, которые не использовались при обучении. В данном случае под корректностью понимается выбор правильных действий. С точки зрения рационального агента – это действия, максимизирующие функцию полезности.

Все классические методы машинного обучения могут использоваться при обучении в мультиагентных системах. Из всех существующих подходов выделим только один – N-уровневых агентов: агент уровня 0 считает, что других агентов нет и просто происходит взаимодействие с окружающей средой; агент уровня 1 считает, что другие агенты есть и они вероятностные (с заданной вероятностью выбирают то или иное действие); агент уровня 2 считает, что все другие агенты уровня 1 и т. д. Важно, что агент уровня $N + 1$ ведет себя всегда эффективней с точки зрения достижения своих целей, чем агент уровня N . Соответственно, при проектировании агента следует решить, на какой уровень противника следует ориентироваться.

Для машинного обучения требуется большое количество экспериментов, на основе которых будет проводиться обучение. Следует учитывать, что в среде виртуального футбола нет возможности ускорить время проведения эксперимента, но есть возможность сохранить и повторно «проиграть» игру тайм за таймом.

4.4. Проблема распределения задач

В классических мультиагентных системах агенты могут взаимодействовать друг с другом. Для реализации взаимодействия разработаны такие спецификации как FIPA ACL (Agent Communication Language) и FIPA Ontology Service Specification. Одной из наиболее известных реализаций ACL является KQML (Knowledge Query and Manipulation Language). Для передачи информации может использоваться KIF (Knowledge Interchange Format).

Каким бы образом не описывались сообщения, они передаются для решения следующих задач: задать вопрос, передать информацию, передать требование, ответить. В случае когда одна и та же задача может быть выполнена несколькими агентами, возникает проблема распределения задач. В этой ситуации агенты должны договориться, кто и какую задачу решает. В мультиагентных системах агенты могут спорить, торговаться, делать контрпредложения, обманывать друг друга, голосовать, устраивать аукционы. В зависимости от назначения могут использоваться различные подходы. Например, один агент может дать другому указание выполнить какую-то задачу, либо устроить аукцион на выполнение задачи. Аукционы подразумевают покупку

чего-либо за деньги, в мультиагентной среде часто встречаются аналоги денег, которыми обмениваются агенты для вознаграждения за решение каких-либо задач.

Все эти подходы и реализации полезны при возможности гарантированной доставки сообщений между агентами. В виртуальном футболе передача аудиосообщения возможна только на ограниченном расстоянии, при этом нет гарантии его доставки, тем не менее проблема распределения задач должна быть решена.

4.5. Спецификации FIPA

В середине 90-х годов была создана организация FIPA (The Foundation for Intelligent Physical Agents), нацеленная на стандартизацию агентных технологий, разработаны соответствующие спецификации построения агентных систем и программных платформ. В частности, сформулированы рекомендации по созданию системы управления агентами, службы каталогов и системы транспортировки сообщений. Одна из возможных реализаций – JADE (Java Agent DEvelopment Framework), реализованная в соответствии со спецификациями FIPA.

Создание решений на основе спецификаций FIPA позволяет создать платформу для разработки, отладки и использования интеллектуальных агентов, но при этом все вопросы реализации особенностей взаимодействия агентов между собой, физические и другие процессы придется реализовывать самостоятельно. В отличие от JADE платформа виртуального футбола предоставляет разработчику среду с заданным интерфейсом взаимодействия, с предопределенными правилами игры и реалистичной имитацией физических процессов.

4.6. Вопросы для самопроверки

1. Что такое интеллектуальный агент?
2. Что такое функция полезности? В чем особенность рационального агента?
3. Какие аспекты следует учитывать при проектировании агента? Какие из них применимы в условиях виртуального футбола?
4. Чем отличаются нормальная, расширенная и характеристическая формы игры?

5. Объясните понятие N-уровневого агента.
6. Для чего используется общение в мультиагентных системах?
7. Для чего была создана FIPA и какими основными ее результатами можно воспользоваться?

Практическая работа 1. ОПРЕДЕЛЕНИЕ МЕСТОНАХОЖДЕНИЯ ИГРОКА НА ПОЛЕ

1.1. Цель и задачи

Целью работы является решение задачи позиционирования агента в пространстве с учетом погрешности получения информации из окружающей среды.

Для достижения поставленной цели необходимо решить следующие задачи:

- подключиться к серверу игры и передать начальные координаты игрока;
- проанализировать ответы сервера;
- вывести в консоль вычисленные координаты игрока.

1.2. Основные теоретические сведения

В разделе «Интерфейс взаимодействия с сервером» на рисунке приведена схема размещения флагов. Пример сообщения от сервера, в котором представлено, что видит игрок:

```
(see 6 ((f c) 15 -19 0 0) ((f r b) 75.9 8) ((f g r b) 68 -13)
((g r) 67.4 -19) ((f g r t) 68 -24) ((f p r b) 54.6 3) ((f p r
c) 50.9 -19) ((f p r t) 54.6 -40) ((f b r 10) 46.5 39) ((f b r
20) 52.5 30) ((f b r 30) 59.7 22) ((f b r 40) 67.4 17) ((f b r
50) 75.9 12) ((f r 0) 72.2 -19) ((f r t 10) 73 -26) ((f r t 20)
75.2 -34) ((f r t 30) 78.3 -41) ((f r b 10) 73 -11) ((f r b 20)
75.2 -3) ((f r b 30) 78.3 4) ((b) 14.9 -19 0 0) ((l r) 71.5 71))
```

Для примера рассмотрим флаг (f c). Известны координаты флага, в сообщении see есть данные о расстоянии до него и соответствующем угле (например, для флага «fc»: расстояние 15, угол -19°). Используя эту информацию, можно вычислить координаты игрока (здесь следует обратить внимание, что угол задан в градусах, а в большинстве формул требуются радианы). Возможны несколько вариантов вычисления:

– решение геометрической задачи (дорисовать треугольники, спроецировать соответствующие перпендикуляры на оси x и y , вычислить координаты пересечений);

– сформировать и обучить нейронную сеть определению координат (входы – данные о расстоянии до всех флагов, а также признак по каждому флагу «виден» или «не виден», выходы – координаты по осям x и y);

– решить систему уравнений с использованием теоремы Пифагора.

Определение координат игрока по двум флагам. Рассмотрим третий подход к решению задачи. Обозначим расстояние до первого флага d_1 , его координаты (x_1, y_1) , для второго флага – d_2 и (x_2, y_2) соответственно. Тогда для поиска координат игрока (x, y) можно составить следующую систему уравнений:

$$\begin{cases} d_1^2 = (x - x_1)^2 + (y - y_1)^2; \\ d_2^2 = (x - x_2)^2 + (y - y_2)^2. \end{cases}$$

Для ее решения следует вычесть одно уравнение из другого:

$$d_1^2 - d_2^2 = (x - x_1)^2 - (x - x_2)^2 + (y - y_1)^2 - (y - y_2)^2.$$

В результате можно выразить x через y :

$$x = y \frac{y_1 - y_2}{x_2 - x_1} + \frac{y_2^2 - y_1^2 + x_2^2 - x_1^2 + d_1^2 - d_2^2}{2(x_2 - x_1)} = \alpha y + \beta.$$

Полученное значение x подставляется в первое равенство:

$$d_1^2 = (\alpha y + \beta - x_1)^2 + (y - y_1)^2.$$

В результате получается квадратное уравнение:

$$(\alpha^2 + 1)y^2 - 2(\alpha(x_1 - \beta) + y_1)y + (x_1 - \beta)^2 + y_1^2 - d_1^2 = 0.$$

Для сокращения записи следует ввести обозначения:

$$a = (\alpha^2 + 1);$$

$$b = -2(\alpha(x_1 - \beta) + y_1);$$

$$c = (x_1 - \beta)^2 + y_1^2 - d_1^2.$$

Далее вычисляются x и y :

$$y = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a};$$

$$x = x_1 \pm \sqrt{d_1^2 - (y - y_1)^2}.$$

В результате будет получено четыре решения, хотя верно только одно. Следует выбрать допустимые координаты (не за пределами поля). При нали-

ции третьего флага f_3 с координатами (x_3, y_3) и расстоянием до него d_3 можно решить задачу алгебраически. Следует вычислить ошибку уравнения $d_3^2 = (x - x_3)^2 + (y - y_3)^2$ для всех решений и выбрать то решение, для которого ошибка меньше $\min(|(x - x_3)^2 + (y - y_3)^2 - d_3^2|)$.

Следует учесть исключительные ситуации:

– если x_1 равно x_2 , тогда значение y может быть вычислено следующим образом:

$$y = \frac{y_2^2 - y_1^2 + d_1^2 - d_2^2}{2(y_2 - y_1)};$$

– если y_1 равно y_2 , тогда значение x может быть вычислено следующим образом:

$$x = \frac{x_2^2 - x_1^2 + d_1^2 - d_2^2}{2(x_2 - x_1)}.$$

Определение координат игрока по трем флагам. В случае наблюдения трех флагов можно свести задачу к решению системы линейных уравнений. Система для первых двух флагов выглядит следующим образом:

$$\begin{cases} d_1^2 = (x - x_1)^2 + (y - y_1)^2; \\ d_2^2 = (x - x_2)^2 + (y - y_2)^2. \end{cases}$$

Из нее можно получить следующую зависимость x от y :

$$x = y \frac{y_1 - y_2}{x_2 - x_1} + \frac{y_2^2 - y_1^2 + x_2^2 - x_1^2 + d_1^2 - d_2^2}{2(x_2 - x_1)} = \alpha_1 y + \beta_1.$$

Система с учетом третьего флага выглядит следующим образом:

$$\begin{cases} d_1^2 = (x - x_1)^2 + (y - y_1)^2; \\ d_3^2 = (x - x_3)^2 + (y - y_3)^2. \end{cases}$$

Из нее можно получить следующую зависимость x от y :

$$x = y \frac{y_1 - y_3}{x_3 - x_1} + \frac{y_3^2 - y_1^2 + x_3^2 - x_1^2 + d_1^2 - d_3^2}{2(x_3 - x_1)} = \alpha_2 y + \beta_2.$$

В результате подстановки получаются следующие равенства:

$$y = \frac{\beta_1 - \beta_2}{\alpha_2 - \alpha_1};$$

$$x = \alpha_1 \frac{\beta_1 - \beta_2}{\alpha_2 - \alpha_1} + \beta_1.$$

Определение координат объекта на поле. Описанные формулы позволяют вычислить координаты (x, y) игрока на поле, однако для решения задач игроку может потребоваться определение координат других объектов (игроков своей и чужой команд, мяча). Координаты любого объекта могут быть вычислены по той же схеме, но в качестве ориентиров будут выступать не два флага, как это описано ранее, а игрок и любой из флагов. При этом известны координаты флага (x_1, y_1) , расстояние до него d_1 , угол α_1 , под которым он виден, вычислены координаты игрока (x, y) , расстояние до объекта d_a и угол α_a , под которым виден объект. Для применения описанных ранее решений необходимо вычислить расстояние от объекта до флага, это может быть выполнено с использованием теоремы косинусов:

$$d_{a1}^2 = d_1^2 + d_a^2 - 2d_1d_a\cos(|\alpha_1 - \alpha_a|).$$

В результате формируется система уравнений:

$$\begin{cases} d_a^2 = (x - x_a)^2 + (y - y_a)^2; \\ d_{a1}^2 = (x_1 - x_a)^2 + (y_1 - y_a)^2. \end{cases}$$

Данную систему уравнений можно решить, используя те же подходы, что и при вычислении координат игрока, но в данном случае в качестве искомого выступают (x_a, y_a) .

1.3. Вопросы для самопроверки

1. Какая информация есть у игрока о видимых флагах в сообщении see?
2. Что еще известно игроку о каждом из флагов?
3. Как игрок может определить свои координаты на поле?
4. Почему координаты игрока вычисляются с использованием трех, а не двух флагов?
5. Какая информация есть у игрока о видимых объектах (мяче, игроках) в сообщении see?
6. Как игрок может определить координаты видимых объектов (мяча, игроков)? Сколько для этого нужно флагов?
7. Для чего предлагается использовать теорему косинусов?

1.4. Задание

Необходимо разработать программу, имитирующую игрока виртуального футбола. Программа должна решать следующие задачи:

1. Получить из командной строки начальные координаты игрока и число (скорость вращения) и командой `move` переместить игрока в заданные координаты.

2. После получения сообщения о начале игры начать вращать игрока с заданной скоростью (поворот на каждом такте). Следует отметить, что движение игрока начнется только после команды рефери `play_on`.

3. По информации, полученной от сервера (сообщение `see`) на каждом такте, вычислить координаты игрока и вывести их в консоль (игрок не должен заранее знать свои координаты, они должны быть вычислены по ориентирам).

4. Разместить на поле дополнительного игрока из команды противника.

5. В процессе вычисления координат игрока (п. 3) одновременно вычислять координаты игрока противника (если он находится в поле зрения).

Практическая работа 2. ДВИЖЕНИЕ ИГРОКА ПО МАРШРУТУ

2.1. Цель и задачи

Целью работы является решение задачи целенаправленного движения агента с использованием видимых ориентиров.

Для достижения поставленной цели необходимо решить следующие задачи:

- подключиться к серверу игры и передать начальные координаты игрока;
- проанализировать ответы сервера;
- определить направление движения, дать соответствующие команды серверу.

2.2. Основные теоретические сведения

Движение игрока по полю начинается после команды рефери `play_on`, которую может услышать игрок (например, `(hear 100 referee play_on)`). Сообщение о забивании гола начинается с `«goal_»`, затем через знак подчеркивания следует, кто забил и номер гола (например, голы в правые ворота `(hear 602 referee goal_1_1)` или `(hear 1176 referee goal_1_2)`).

Формирование списка флагов, с которыми осуществляется работа, аналогично списку флагов из практической работы 1, но следует учесть, что игрок в том числе будет интересоваться мяч, который обозначается `(b)`.

Обработку сообщения от сервера и формирование команды следует выполнять в объекте-контроллере (Controller). Данный объект должен в качестве свойств хранить массив с требуемой последовательностью действий и номер действия в данной последовательности. Изменение номера действия означает переход к следующему действию. Каждое действие по движению к флагу может выглядеть, например, следующим образом {act: "flag", fl: "frb"}, имя флага хранится в поле fl; действие kick предполагает удар по мячу, в этом случае необходима цель, например середина ворот: {act: "kick", fl: "b", goal: "gr"}. Поле goal объекта хранит имя флага (середина ворот).

Для обработки сообщения от сервера с учетом целевого действия потребуется обработка того, что видит и слышит игрок. Прежде всего движение предполагается только после соответствующей команды рефери, после этого необходимо проанализировать весь список флагов, который виден игроку. При этом возможны три ситуации при движении к флагу (алгоритм):

1. Искомый флаг не виден, значит игроку следует повернуться в поисках искомого флага.

2. Искомый флаг виден и находится далеко от игрока, тогда необходимо повернуться и двигаться в направлении флага.

3. Искомый флаг виден и находится близко от игрока, тогда необходимо либо уменьшить скорость приближения, либо переходить к следующему действию в массиве действий (предлагается переходить к следующему действию, если расстояние до флага в маршруте движения меньше 3).

При выполнении удара по мячу возможны следующие ситуации:

1. Мяч не виден, значит игроку следует повернуться в поисках мяча.

2. Мяч виден и находится далеко – движение к мячу по аналогии с движением к флагу, но целевое расстояние до мяча меньше – 0.5.

3. Мяч находится в зоне возможного удара, но ворота не видны, тогда необходимо с небольшой силой ударить мяч, чтобы он откатился вбок (например, на 45° вправо (kick 10 45)).

4. Мяч находится в зоне возможного удара и ворота видны, тогда следует сильно ударить мяч в направлении ворот (например, если ворота прямо по направлению движения, то (kick 100 0)).

Соответственно, предлагается следующая реализация:

1. Анализ того, что видит игрок.

2. Проверка, что текущее действие выполнено, при необходимости – переход к следующему действию.

3. Формирование команды серверу для выполнения действия.

2.3. Вопросы для самопроверки

1. Какой формат у аудиосообщений `hear`?

2. Опишите алгоритм действий при движении от одного флага к другому флагу. Как называются и где находятся использованные флаги?

3. Опишите алгоритм действий при ударе по мячу.

2.4. Задание

Необходимо разработать программу, имитирующую игрока виртуального футбола. Программа должна решать следующие задачи:

1. В программе имитации задана последовательность действий игрока (например, `[{act: "flag", fl: "frb"}, {act: "flag", fl: "gl"}, {act: "flag", fl: "fc"}, {act: "kick", fl: "b", goal: "gr"}]`).

2. Программа должна выполнить заданную последовательность действий:

- при выполнении действия «движение к флагу» флаг считается достигнутым, если расстояние до флага меньше 3;
- действие «удар по мячу» должно выполняться до забивания гола;
- после забивания гола последовательность действий (п. 1) должна повториться с начала.

Практическая работа 3. УПРАВЛЕНИЕ НЕСКОЛЬКИМИ ИГРОКАМИ

3.1 Цель и задачи

Целью работы является решение задачи координации действий автономных агентов с использованием деревьев решений.

Для достижения поставленной цели необходимо решить следующие задачи:

- разработать и подключить механизм обработки деревьев решений;
- разработать деревья решений для управления агентами.

3.2. Основные теоретические сведения

Деревья решений (деревья принятия решений) представляют собой структуры, состоящие из узлов, ветвей и листьев. В узлах дерева осуществляется проверка условий (переменных, сформированных на основании данных из окружающей среды). В зависимости от результатов проверки выбираются соответствующие ветви дерева. В простейшем случае это может быть бинарное дерево, выбор левой ветви в случае истинности условия в узле, выбор правой ветви – в случае ложности условия в узле. В листьях деревьев хранятся решения, применительно к виртуальному футболу в листьях формируются команды, передаваемые на сервер.

Деревья решений являются одним из простейших способов формализации алгоритмов и могут быть наглядно изображены графически, что существенно упрощает их анализ. Функции условий в узлах ветвления часто пишут на скриптовых языках, что увеличивает гибкость реализации.

Рассмотрим возможный вариант представления деревьев решений. В нем предусмотрены следующие структуры:

- хранение информации о текущем состоянии дерева *store*;
- корень дерева *root*;
- исполняемые узлы дерева *exec-next*, в котором функция *exec* вычисляет необходимые значения, сохраняет их в *store* для последующей обработки в узлах-условиях;

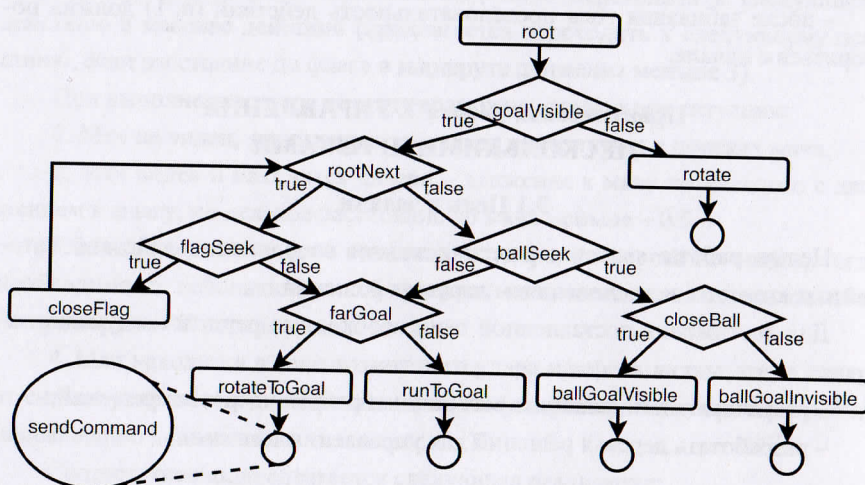


Рис. 3.1. Дерево решений для забивания гола

– узлы-условия `condition-trueCond-falseCond` проверяют простые условия и в зависимости от истинности или ложности результата осуществляют переход к одной или альтернативной ветви в дереве;

– узлы-команды `command`, которые возвращают команду на исполнение.

Дерево решений одного игрока. Рассмотрим пример дерева решений, предполагающее движение по маршруту с забиванием мяча в правые ворота (рис. 3.1).

```
const FL = "flag", KI = "kick"
const DT = {
  state: {
    next: 0,
    sequence: [{act: FL, fl: "frb"}, {act: FL, fl: "gl"},
               {act: KI, fl: "b", goal: "gr"}],
    command: null
  },
  root: {
    exec(mgr, state) { state.action =
state.sequence[state.next]; state.command = null },
    next: "goalVisible",
  },
  goalVisible: {
    condition: (mgr, state) => mgr.getVisible(state.action.fl),
    trueCond: "rootNext",
    falseCond: "rotate",
  },
  rotate: {
    exec (mgr, state) { state.command = {n: "turn", v: "90"} },
    next: "sendCommand",
  },
  rootNext: {
    condition: (mgr, state) => state.action.act == FL,
    trueCond: "flagSeek",
    falseCond: "ballSeek",
  },
  flagSeek: {
    condition: (mgr, state) => 3 >
mgr.getDistance(state.action.fl),
    trueCond: "closeFlag",
    falseCond: "farGoal",
  },
  closeFlag: {
    exec(mgr, state)
    {state.next++; state.action = state.sequence[state.next]},
    next: "rootNext",
  },
  farGoal: {
    condition:
```



```

    (mgr, state) => mgr.getAngle(state.action.fl) > 4,
    trueCond: "rotateToGoal",
    falseCond: "runToGoal",
  },
  rotateToGoal: {
    exec (mgr, state) { state.command = {n: "turn", v:
mgr.getAngle(state.action.fl)} },
    next: "sendCommand",
  },
  runToGoal: {
    exec (mgr, state) { state.command = {n: "dash", v: 100} },
    next: "sendCommand",
  },
  sendCommand: {
    command: (mgr, state) => state.command
  },
  ballSeek: {
    condition:
      (mgr, state) => 0.5 > mgr.getDistance(state.action.fl),
    trueCond: "closeBall",
    falseCond: "farGoal",
  },
  closeBall: {
    condition:
      (mgr, state) => mgr.getVisible(state.action.goal),
    trueCond: "ballGoalVisible",
    falseCond: "ballGoalInvisible",
  },
  ballGoalVisible: {
    exec (mgr, state) { state.command =
      {n: "kick", v: `100 ${mgr.getAngle(state.action.goal)}`},
    next: "sendCommand",
  },
  ballGoalInvisible: {
    exec (mgr, state) {state.command = {n: "kick", v: "10 45"}},
    next: "sendCommand",
  },
}

```

В приведенном примере состояние state хранит три переменные: номер в массиве целей, массив целей (как в практической работе 2) и команду.

Узел root инициализирует переменную state.action текущей целью, «сбрасывает» текущую команду и переходит к узлу goalVisible, в котором проверяется условие – виден ли флаг state.action, если виден – переход к rootNext, иначе – rotate.

Узел rotate подготавливает команду поворота на 90° и переходит к узлу sendCommand, который передаст данную команду на исполнение.

Узел `rootNext` проверяет, осуществляется ли движение к флагу или требуется удар по мячу, в первом случае – переход к узлу `flagSeek`, во втором – к узлу `ballSeek`.

В узле `flagSeek` проверяется расстояние до флага, в случае близкого расстояния – переход к узлу `closeFlag`, большого расстояния – `farGoal`. В случае `closeFlag` выполняется переход к следующей цели в массиве целей (флаг достигнут) и переход к узлу `rootNext`.

В узле `farGoal` проверяется угол, под которым видна цель; если он большой, то переход к узлу `rotateToGoal`, иначе – к узлу `runToGoal`.

Узел `rotateToGoal` формирует команду поворота к цели, узел `runToGoal` формирует команду движения к цели.

Узел `ballSeek` проверяет расстояние до мяча; в случае маленького расстояния – переход к узлу `closeBall`, иначе – `farGoal`.

В узле `closeBall` проверяется, видны ли ворота, в которые необходимо направить мяч, если видны, то переход к узлу `ballGoalVisible`, иначе – к узлу `ballGoalInvisible`. В обоих случаях формируется команда удара по мячу, но под разными углами.

Команды отправляются с использованием `sendCommand`.

Важное ограничение предложенного дерева – поиск целевого объекта в случае, когда он не виден, – реализован просто поворотом на 90° , при этом не учитывается, какие флаги видит игрок, где он в последний раз видел искомым объект.

Следует отметить, что формально приведенный ориентированный граф не является деревом решений, так как в нем есть циклы (переход от `closeFlag` к `rootNext`). В классическом варианте следовало бы сформировать дерево без циклов, но тогда увеличится размер графа и могут появиться повторы.

Менеджер-обработчик дерева решений. Для автоматизации обработки деревьев решений используется дополнительный объект-менеджер `mgr`, который выполняет рутинные операции по определению видимости объекта на поле `getVisible()`, расстояния до него `getDistance()`, угла `getAngle()` и т. п. Соответственно, на каждом шаге такой менеджер должен быть проинициализирован тем, что видит игрок. Основная функция менеджера должна обеспечить исполнение дерева решений и формирование команды. Пример такой функции:


```

getAction(dt, p) {
  this.p = p
  function execute(dt, title) {
    const action = dt[title]
    if(typeof action.exec == "function") {
      action.exec(Manager, dt.state)
      return execute(dt, action.next)
    }
    if(typeof action.condition == "function") {
      const cond = action.condition(Manager, dt.state)
      if(cond)
        return execute(dt, action.trueCond)
      return execute(dt, action.falseCond)
    }
    if(typeof action.command == "function") {
      return action.command(Manager, dt.state)
    }
    throw new Error(`Unexpected node in DT: ${title}`)
  }
  return execute(dt, "root")
}

```

В приведенном примере функция `getAction` получает в качестве параметров `dt` – дерево решений и `p` – массив информации, полученной с сообщением `see`. Предполагается, что `getAction` – функция объекта, поэтому массив `p` сохраняется для последующей обработки в функциях `getVisible`, `getDistance` и т. д. Функция `execute` в качестве параметра получает обрабатываемое дерево и узел `root` для обработки. Обработка осуществляется рекурсивно, каждая функция дерева возвращает результирующую команду. Возможен вызов трех функций `exec`, `condition` и `command`. Все три в качестве параметров получают текущий объект и `state` дерева. Проверка принадлежности узла тому или другому виду осуществляется по наличию соответствующих функций с использованием `typeof`.

Дерево решений для двух игроков. Универсальное дерево решений для двух игроков, когда один игрок движется впереди, а второй слева сзади, заключается в следующем:

1. Вычислить текущую цель.
2. Если количество видимых игроков своей команды равно нулю, то воспользоваться деревом движения по флагам.
3. Если количество видимых игроков своей команды равно одному, то п. 3.1 (фрагмент алгоритма для «иначе» – в рамках псевдокода не приведен).
- 3.1. Определим переменные: `dist` – расстояние до игрока, `angle` – угол, под которым виден игрок.

3.2. Если $\text{dist} < 1$ and $|\text{angle}| < 40$, тогда turn на 30, иначе п. 3.3.

3.3. Если $\text{dist} > 10$, то п. 3.3.1, иначе п. 3.4.

3.3.1. Если $|\text{angle}| > 5$, то turn на angle, иначе dash на 80.

3.4. Если $\text{angle} > 40$ or $\text{angle} < 25$, то turn на $\text{angle}-30$, иначе п. 3.5.

3.5. Если $\text{dist} < 7$, то dash на 20, иначе dash на 40.

Основные идеи приведенного алгоритма для двух игроков следующие:

- если игрок не видит других игроков (п. 2), значит он впереди, а ведомый игрок с тем же деревом решений – сзади;

- если расстояние слишком маленькое (п. 3.2), значит необходимо избежать столкновения (данная ситуация может возникнуть при развороте у флага);

- если расстояние очень большое (п. 3.3), то нужно повернуться в сторону цели и быстро к ней приближаться;

- если расстояние небольшое, то нужно повернуться так, чтобы угол на цель был примерно 30° и двигаться со скоростью 40, а при сильном приближении – 20.

Основным недостатком данного алгоритма является интеграция с деревом движения по флагам, когда достигается флаг, то осуществляется разворот первого игрока, за счет чего меняются позиции в команде (ведущий на ведомый). Вторым недостатком предложенного алгоритма является то, что не учитывается достижение общей цели первым игроком. При выполнении практических работ от этих недостатков предлагается избавиться.

Дерево решений вратаря. Основные идеи алгоритма для вратаря (в воротах справа) в случаях, когда мяч не виден или он далеко, следующие:

- если виден флаг gr и дистанция до него больше 5, то нужно к нему приблизиться;

- примерное расстояние до флага $fprc$ от 12 до 16, до $fprl$ и $fprb$ от 20 до 28. Команда dash поддерживает отрицательные параметры (можно двигаться спиной вперед, но с небольшой скоростью);

- заняв позицию в центре ворот, следует повернуться в сторону мяча.

Основные идеи алгоритма для вратаря (в воротах справа) в случаях, когда мяч близко, следующие:

- на небольшом расстоянии (до 2) движущийся мяч можно поймать (catch), в остальных случаях его можно пнуть;

– сильно мяч пинать следует в сторону ворот противника (флаги `gl`, `flt`, `flb`), в остальных случаях рекомендуется или несильно пнуть мяч, или вернуться для повышения вероятности попадания в ворота противника;

– после отдаления мяча на достаточное расстояние вратарь должен возвращаться в ворота.

3.3. Вопросы для самопроверки

1. Что такое деревья решений и для чего они нужны? Какие в них бывают узлы? Сколько ветвей может быть в узлах ветвления?

2. Какие типы узлов предлагается использовать при описании деревьев решений для виртуального футбола?

3. Для чего нужен менеджер деревьев решений? Какой алгоритм его работы?

4. Какие функции предложено реализовать в менеджере деревьев решений?

3.4. Задание

Необходимо разработать с использованием деревьев решений две программы (два дерева), имитирующие игрока и вратаря виртуального футбола.

Программа игрока должна решать следующие задачи:

1. Формирование «звена» из двух (трех) игроков. Для «звена» из двух игроков один впереди, второй чуть сзади слева, для трех игроков: один впереди, двое сзади (справа и слева). Решение о месте игрока в «звене» должно выбираться автономно каждым игроком.

2. Движение игроков в составе «звена» по заданному маршруту (последним пунктом маршрута – забивание гола в ворота справа).

Программа вратаря должна решать следующие задачи:

1. Защита ворот на правой половине поля.

2. Отбивание `kick` и поимка мяча `catch`.

Практическая работа 4. КООРДИНАЦИЯ ДЕЙСТВИЙ ИГРОКОВ

4.1. Цель и задачи

Целью работы является решение задачи координации действий игроков одной команды для обеспечения голевого паса.

Для достижения поставленной цели необходимо решить следующие задачи:

- создать программу игрока, обеспечивающего передачу паса;
- создать программу игрока, обеспечивающего забивание гола, после переданного паса, с учетом возможных погрешностей в передаче паса;
- обеспечить многократное повторение голевого паса в течение игры.

4.2. Основные теоретические сведения

Аудиосообщения. Информация об изменениях в ходе игры предоставляется рефери в виде аудиосообщений `hear`. Формат сообщения: (`hear` Время Источник Текст). Здесь Время – такт, на котором передано сообщение, Источник – отправитель сообщения (если игрок сам отправил данное сообщение, то Источник будет `self`), Текст – текст сообщения. Важно, что текст сообщения, переданный игроком `say`, будет заключен в кавычки:

```
(say go)
(hear 100 self "go")
(hear 110 referee play_on)
```

Аудиосообщения могут использоваться для координации действий игроков, если они находятся не слишком далеко друг от друга. В частности, аудиосообщение рефери `play_on` гласит, что игра начата (в самом начале игры и после забитого мяча), а сообщение `goal_1_2` гласит, что команда слева забила второй гол.

Соответственно, агент может в дереве решений учитывать полученные аудиосообщения. Например, при получении сообщения `play_on` можно начинать движение (`play_on` в течение игры может быть отправлен несколько раз), а после получения сообщения, начинающегося на «`goal_`», может с использованием команды (`move x y`) переместиться в начальное положение (`x, y`) на поле.

В начале игры (перед получением сообщения `play_on`) и после каждого забитого гола (после «`goal_...`») мяч оказывается в центре поля, что позволяет игрокам начать целевую последовательность действий для передачи голевого паса и забивания гола с начала.

При передаче голевого паса может сложиться ситуация, когда игрок окажется близко к воротам противника в отсутствие игроков противника, что может привести к состоянию вне игры (`offside`). В связи с этим целесообразно «установить» двух противников у ворот, например около каждой штанги. Эти два игрока противника должны стоять на месте независимо от сообщений сервера.